System Verification and Validation Plan for Flexin'

Team 3, Flexin'
Arjun Karthik
Shao (Daniel) Hu
Noah Goldschmied
Peter Deforest
Daniel Franze-Da Silva

November 1, 2026

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 3
- 2.4 Relevant Documentation . . . 3

3 Plan **4**
- 3.1 Verification and Validation Team . . . 4
- 3.2 SRS Verification . . . 6
- 3.3 Design Verification . . . 6
- 3.4 Verification and Validation Plan Verification . . . 7
- 3.5 Implementation Verification . . . 7
- 3.6 Automated Testing and Verification Tools . . . 7
- 3.7 Software Validation . . . 8

4 System Tests **8**
- 4.1 Tests for Functional Requirements . . . 8
  - 4.1.1 Account Management . . . 8
  - 4.1.2 Fitness Tracking and Goals . . . 9
  - 4.1.3 Rewards and Virtual Currency . . . 9
  - 4.1.4 Simulated Investing . . . 9
  - 4.1.5 Community and Leaderboards . . . 12
  - 4.1.6 Progress Reporting . . . 14
- 4.2 Tests for Nonfunctional Requirements . . . 15
  - 4.2.1 Look and Feel . . . 15
  - 4.2.2 Usability and Humanity . . . 16
  - 4.2.3 Performance . . . 16
  - 4.2.4 Operational and Environmental . . . 31
  - 4.2.5 Maintainability and Support . . . 31
  - 4.2.6 Security . . . 32
  - 4.2.7 Cultural . . . 40
  - 4.2.8 Compliance . . . 40
  - 4.2.9 User Documentation and Training . . . 41
- 4.3 Traceability Between Test Cases and Requirements . . . 44

5 Unit Test Description **45**
5.1 Unit Testing Scope . . . 46
5.2 Tests for Functional Requirements . . . 46
5.2.1 Module 1 . . . 46
5.2.2 Module 2 . . . 47
5.3 Tests for Nonfunctional Requirements . . . 47
5.3.1 Module ? . . . 47
5.3.2 Module ? . . . 48
5.4 Traceability Between Test Cases and Modules . . . 48

6 Appendix **49**
6.1 Symbolic Parameters . . . 49
6.2 Usability Survey Questions? . . . 49

7 Appendix — Generative AI Use Disclosure **50**
7.1 AI Tools Used . . . 50
7.2 How and Where Used . . . 50
7.3 Verification of AI-Generated Content . . . 50

8 Appendix — Reflection **51**

List of Tables

1 Verification and Validation Team Roles . . . 5
[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

[symbols, abbreviations, or acronyms — you can simply reference the SRS (**?**) tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

2.1 Summary

The software being tested is Flexin', an app that allows users to learn about the power of compounding through working out and investing. The team will be looking to test the following functions of the app:

- Step tracking and integrations with fitness data sources (Android Health Connect and, optionally, Garmin watches)
- Real-time updates of fitness data within the app
- Converting fitness progress into virtual currency and depositing it into the user's account
- Viewing account balances
- Simulated/generated financial markets
- Performance leaderboards

2.2 Objectives

The main objective of the VnV plan is to ensure that Flexin' properly teaches the impact of compounding. For the VnV plan to achieve this objective, the team will prioritize the following qualities:

- **Learnability:** The app is easy for new users to learn, as measured by the learnability requirement in the SRS (UHR-EOU-08).
- **Correctness of currency calculations:** Virtual currency earned, deposited, and displayed in account balances is calculated correctly, and operations that change balances either complete fully or are rolled back (PR-RFT-02).

- **Consistency of asset calculations:** Simulated asset values are consistent and produce the returns expected from the implemented market algorithms, including the calibration against historical market data required by PR-PA-06.

- **User confidence in the financial lessons:** Users come away trusting and understanding the financial lessons the app is intended to convey.

The following objectives are out of scope for this VnV plan:

- **Assumptions in the SRS and Hazard Analysis:** The assumptions listed in the SRS (**?**) and the Critical Assumptions section of the Hazard Analysis are taken as given and will not be verified.

- **Third-party packages and libraries:** External dependencies, such as Android Health Connect and the Garmin Connect IQ SDK, are assumed to be correct and already verified by their implementation teams (e.g. Garmin). The team does not have the time or resources to verify these libraries within the scope of this capstone. This matches the Hazard Analysis assumption that third-party health platforms keep their documented behaviour. Flexin''s own handling of outages and malformed data from these sources is still in scope (PR-RFT-03).

- **Internet connectivity:** The SRS lists a mobile data or Wi-Fi connection as part of the environment needed to reach the Flexin' backend. The strength and reliability of the user's connection will not be tested. The app's required offline behaviour (PR-RFT-01 and PR-RFT-04) is still in scope.

- **Full verification of user-reported fitness activity:** The team will not attempt to fully verify that users are not lying about completing fitness goals, as this would require too much time and resources. The SRS already assumes that fitness devices report correct data and that artificial changes are dishonest use, and SC-04 limits this risk by not allowing manual entry of activity. Small checks may be used to gauge validity in some cases (e.g. checking that a Garmin watch's heart rate reading stays above a threshold for a period of time), but validity cannot be guaranteed, especially for users without a Garmin device who track activity solely through their phone.

2.3 Custom Documents

The team is completing two custom documents (CDs), one per term:

- **CD-Fall — User/Stakeholder/Client Interview Report:** A report on interviews with the project's client and potential users, used to elicit requirements. The SRS appendix already summarizes the two interviews conducted so far: a mock interview with the client and supervisor, and an interview with a representative end user.
- **CD-Winter — Business Plan:** A business plan exploring how Flexin' could be monetized after the capstone, as the team believes the project has the potential to become a startup. It will cover the market need, market size and business model, costs, competition, and an execution plan.

2.4 Relevant Documentation

The following documents are relevant to the VnV efforts:

- **Software Requirements Specification (SRS)** (**?**): Defines the functional and non-functional requirements of Flexin', each with a fit criterion. These fit criteria are what the tests in this plan check against. The SRS also lists the assumptions that bound what is in scope for verification, and names the testing tools the team will use (JUnit for backend unit and system tests, and Detox for end-to-end tests of the mobile client).
- **Hazard Analysis:** Identifies the hazards of using Flexin' and defines twelve safety and security requirements (SR1–SR12) to mitigate them, each with a fit criterion. Its roadmap commits to implementing SR1, SR2, SR3, SR6, SR8, SR9, SR11, and SR12 within the capstone, so these are the safety and security requirements this plan verifies. Its Critical Assumptions are excluded from verification.
- **Health Connect and Garmin Connect IQ SDK documentation:** Describes how activity data is provided to Flexin'. Since these platforms are assumed to keep their documented behaviour, their documentation defines the data the integration is expected to receive.

- **Custom Documents:** The interview report (CD-Fall) records the user and client needs that the requirements were derived from, which is what validation testing checks the app against. The business plan (CD-Winter) has little direct bearing on VnV, but one SRS scalability requirement refers to it for how the product might grow after the capstone.
- **User interviews and user testing:** The interviews in the SRS appendix ground the requirements in real user needs. The SRS also plans user testing on demo builds in Term 2 (Weeks 17 and 21), which will be used to assess learnability and user confidence in the financial lessons.

3 Plan

This section outlines the plan to verify and validate Flexin' across its project development lifecycle. The plan will detail the roles and responsibilities of the team, and will specify the approach to verify the SRS document, design and implementation, as well as how to verify the VnV itself. Lastly, it will describe the automated tools and software validation framework that the team plans on using.

3.1 Verification and Validation Team

The process of VnV will be a collective effort spread across the team and supervisor. All members will have defined roles that altogether will cover the process. These roles can be seen in Table 1.

Table 1: Verification and Validation Team Roles

Team Member	**Project Role**	**VnV Responsibilities**
Peter Deforest	Testing Lead	Coordinating all VnV activities, ensuring unit, integration and regression tests are written and maintained, and performing the final review of all test cases before each milestone.
Arjun Karthik	GitHub Manager, CI/CD Lead	Implementing and maintaining the CI/CD pipeline and static analysis tools, monitoring automated test coverage metrics, and resolving issues with VnV tooling.
Shao (Daniel) Hu	Lead Developer	Leading code inspections using a defined checklist, verifying that the design and implementation satisfy the SRS, and leading the code walkthrough for the supervisor.
Daniel Franze-Da Silva	Scribe, UI/UX Design Lead	Documenting VnV meeting minutes, preparing and running the usability survey, and verifying that the UI and UX meet the usability requirements in the SRS.
Noah Goldschmied	Team Liaison, Chairman	Maintaining traceability between requirements and test cases, coordinating SRS and VnV Plan reviews, and ensuring feedback from the supervisor, TAs and peer reviewers is addressed through GitHub issues.
Chris Schankula	Project Supervisor	Providing technical oversight and domain feedback on the fitness and finance features, reviewing the SRS and VnV Plan, and participating in the Revision 0 demonstration to validate the requirements.

3.2 SRS Verification

The SRS will be verified to ensure that all requirements detailed in the document are reasonable, accurate, understandable and complete. The bullet list below describes the different avenues in which the SRS may be reviewed.

- **Submission to TA:** The SRS will be submitted to our course TA, who will thoroughly read and analyze the document and provide feedback where needed.
- **Peer Review:** Classmates will review the SRS document and ensure its content makes sense from a course standpoint.
- **Presentation to Supervisor:** The team may meet with the project supervisor to ensure that the requirements described in the SRS are high quality and in line with what they expect to be delivered.

If any problem are found with the SRS, the team will utilize our issue board to keep track and fix them. Once development begins, these issues may also coincide with a bugfix.

3.3 Design Verification

Design verification will begin after the submission of the Design Document on November 16th. This verification will ensure that Flexin's architecture meets requirements and is implementable. The bullet list below describes how the Design Document may be reviewed.

- **Submission to TA:** The Design Document will be submitted to our course TA, who will thoroughly read and analyze the document and provide feedback where needed.
- **Peer Review:** Classmates will review the Design document and ensure its content makes sense from a course standpoint.
- **Presentation to Supervisor:** The team may meet with the project supervisor to ensure that the design is feasible from an implementation point of view.

Any problems with the Design Document will be handled through the teams issue board.

3.4 Verification and Validation Plan Verification

VnV plan verification will take place before the document is submitted by the team, as well as post submission by other members.

- **Team Review:** The team will review the VnV Plan ensuring that the defined tasks are feasible within the timeframe of the course and that the detail described in the document is appropriate.
- **Peer Review:** Classmates will review the VnV Plan and ensure its content makes sense from a course standpoint.
- **Presentation to Supervisor:** The team may meet with the project supervisor to ensure that the VnV Plan is feasible.
- **Submission to TA:** The VnV Plan will be submitted to the TA for grading and feedback.

3.5 Implementation Verification

Implementation verification ensures that the system, as built, correctly implements the SRS and design document. This will be done through a combination of the following:

- **Unit Testing and System Testing:** All functional and non-functional requirements will be verified through the tests described in the below sections of this document.
- **Static Analysis and Linting:** The team will enforce a coding standard using linters and static analyzers.
- **Code Review:** All code changes will require a pull request and approval from at least one team member. The team members will be looking for code correctness, standards, and proper error handling.

3.6 Automated Testing and Verification Tools

The team plans on creating standard unit tests that will run during the CI stage of the code development lifecycle. The framework used for the backend will most likely be JUnit, and for the frontend, Detox. The team may also create integration tests and end-to-end tests to ensure that application

functionality doesn't degrade in the grand scheme. Code coverage will be top of mind while developing this test plan, and a tool to ensure that a minimum agreed threshold of 80% coverage will be implemented.

3.7 Software Validation

The software validation step of the VnV plan will ensure that the near-final product lives up to its expectations. This validation will be done from the perspective of the users and the stakeholders.

- **Rev-0 Demonstration:** The Rev-0 demo will be used to ensure that the core functionality of Flexin' is up to the standards of what the supervisor expects.
- **Usability Tests:** The team plans on giving Flexin' to a select group of prospective users for a period of roughly 2 weeks. This will help the team understand what needs work, and what worked well for the users.

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

The subsections below group the functional requirements in the Functional Requirements section of the SRS (**?**) by feature area. Together they cover FUNC-01 to FUNC-16. FUNC-17 (Sponsored Reward Redemption) is a stretch goal in the SRS Waiting Room, so it is not tested.

[Each test uses the fields Control, Initial State, Input, Output, Test Case Derivation, and How test will be performed. —SS]

4.1.1 Account Management

These tests cover creating an account and signing in.

SRS ID	Requirement
FUNC-01	User Registration
FUNC-02	User Authentication

[Add the test cases for these requirements. —SS]

4.1.2 Fitness Tracking and Goals

These tests cover importing activity data from third-party providers and creating and tracking fitness goals.

SRS ID	Requirement
FUNC-03	Third-Party Fitness Data Import
FUNC-04	Fitness Goal Creation
FUNC-05	Fitness Goal Tracking

[Add the test cases for these requirements. —SS]

4.1.3 Rewards and Virtual Currency

These tests cover awarding virtual currency for activity and keeping each user's balance.

SRS ID	Requirement
FUNC-06	Reward Allocation
FUNC-07	Virtual Currency Tracking

[Add the test cases for these requirements. —SS]

4.1.4 Simulated Investing

These tests cover the portfolio and trading features of the simulated market.

SRS ID	Requirement
FUNC-08	Portfolio Creation
FUNC-09	Asset Purchase
FUNC-10	Asset Sale
FUNC-11	Transaction Recording
FUNC-12	Portfolio Performance Calculation

test-FR-09	**Asset Purchase**
Type	Dynamic, Automatic
Initial State	The user is logged in and has enough virtual currency to purchase the asset.
Input/Condition	The `UserID`, the `AssetID`, and a call to the buy action.
Output/Result	The cost of the asset is deducted from the user's currency balance, and the asset is added to the user's portfolio.
Covers	FUNC-09. Users can purchase available assets if they have sufficient virtual currency, so the balance must decrease by the asset's cost and the asset must appear in the portfolio.
How test will be performed	After the purchase of an asset, the balance is checked to confirm the currency was removed and the portfolio is checked to confirm the asset was added.

test-FR-10	**Asset Sale**
Type	Dynamic, Automatic
Initial State	The user is logged in and holds at least one previously purchased asset in their portfolio.
Input/Condition	The `UserID`, the `AssetID`, and a call to the sell action.
Output/Result	The sale value is added to the user's currency balance, and the asset is removed from the portfolio.
Covers	FUNC-10. Users can sell assets currently held in their portfolio, so the proceeds must be credited and the holding removed.
How test will be performed	After the sale of an asset, the balance is checked to confirm the currency was added and the portfolio is checked to confirm the asset was removed.

test-FR-11	**Transaction Recording**
Type	Dynamic, Manual
Initial State	The user is logged in and has a non-empty transaction history.
Input/Condition	The `UserID`, and a request to view previous market transactions.
Output/Result	A list of all transactions the user has made, including the type (buy or sell), asset, quantity, price and time.
Covers	FUNC-11, and also confirms the effects of FUNC-09 and FUNC-10. Every purchase and sale generates a transaction record the user can view at any time, so the list must match the trades that were made.
How test will be performed	After completing one or more transactions on an account, the user's transaction list is viewed and checked against the trades that were made.

test-FR-12	Portfolio Performance Calculation
Type	Dynamic, Automatic
Initial State	The user is logged in and has a portfolio containing at least one asset.
Input/Condition	The `UserID`, and a request to retrieve all assets in the portfolio and calculate performance.
Output/Result	The portfolio's current value and its percentage gain or loss. A graph of growth or loss over time is shown to the user.
Covers	FUNC-12. Users can view their current portfolio value and unrealized gains or losses, which helps them judge how well their investment strategies have performed.
How test will be performed	Through manual inspection and the use of financial calculators, the displayed portfolio value, percentage gain or loss, and growth graph are measured and verified.

[Add the test cases for these requirements. —SS]

4.1.5 Community and Leaderboards

These tests cover friends lists and public and private leaderboard groups.

SRS ID	Requirement
FUNC-13	Leaderboard Participation
FUNC-14	Friend Management
FUNC-15	Private Leaderboards

test-FR-13	**Leaderboard Participation**
Type	Dynamic, Manual
Initial State	The user is logged in, and a leaderboard with open spaces is available to join.
Input/Condition	The `UserID` and the `LeaderboardID` of a leaderboard with open spaces.
Output/Result	The user joins the intended leaderboard and appears in its member list.
Covers	FUNC-13. A user can join a group, so after the join request the user must be a member of the chosen leaderboard and able to compete with its other members.
How test will be performed	After the user attempts to join the leaderboard, check that the user's `UserID` is linked to that `LeaderboardID`.

test-FR-14	**Friend Management**
Type	Dynamic, Automatic
Initial State	The user is logged in and can search for other users by `UserID`.
Input/Condition	The `UserID` of another application user.
Output/Result	A friend request is sent to that user, and the invitee can respond to it.
Covers	FUNC-14. Users can add and remove friends, so a request must reach the invitee, and the result of their response (approved or denied) must be reflected in both users' friends lists. This gives users the social aspect of comparing progress with selected friends.
How test will be performed	After the user sends a friend request, check that both the requester and the invitee can see the request and, once the invitee responds, the result of the action (approved or denied).

test-FR-15	Private Leaderboard Creation
Type	Dynamic, Automatic
Initial State	The user is logged in.
Input/Condition	A valid `UserID` and a request to create a new leaderboard.
Output/Result	A new private leaderboard is created with a unique `LeaderboardID`.
Covers	FUNC-15. Users can create private competition groups, so a successful request must produce a new group with its own ID. The created group is also needed by the FR-13 tests, which require a leaderboard to join.
How test will be performed	After the user completes the steps to create a leaderboard, check that a valid `LeaderboardID` exists and that the creator is a member of the new leaderboard.

4.1.6 Progress Reporting

These tests cover the summaries of activity, goals, earnings, portfolio performance, and standings.

SRS ID	Requirement
FUNC-16	Progress Reporting

test-FR-16	Progress Reporting
Type	Dynamic, Automatic
Initial State	The user is logged in and has a trackable history of activity, goals, currency earnings and portfolio activity.
Input/Condition	A valid `UserID` and a request to view the progress data connected to that ID.
Output/Result	Current and historical summaries of the user's fitness activity, goal completion, virtual currency earnings, portfolio performance and leaderboard standings, each matching the data recorded for that user.
Covers	FUNC-16. Users can access current and historical progress information at any time, so each summary must match the stored data. It also verifies the combined effects of FUNC-03, FUNC-04, FUNC-05, FUNC-06, FUNC-07 and FUNC-12, whose data feed the report.
How test will be performed	After the `UserID` is submitted to retrieve account progress, check that the returned summaries include the user's activity, goal tracking, currency earnings, portfolio performance and leaderboard standings if applicable

4.2 Tests for Nonfunctional Requirements

The subsections below follow the nonfunctional requirement sections of the SRS (**?**). Each one lists the SRS subsections and requirement IDs that its tests cover.

[Each test uses the fields Type, Initial State, Input/Condition, Output/Result, and How test will be performed. Static tests, such as reviews, inspections, and walkthroughs, should instead say who is involved and how the review is run. —SS]

4.2.1 Look and Feel

These tests cover the Look and Feel Requirements section of the SRS.

Appearance Covers LF-APP-01 to LF-APP-05.
[Add the test cases for these requirements. —SS]

Style Covers LF-STY-01 to LF-STY-04.
[Add the test cases for these requirements. —SS]

4.2.2 Usability and Humanity

These tests cover the Usability and Humanity Requirements section of the SRS.

Ease of Use Covers UHR-EOU-01 to UHR-EOU-09.
[Add the test cases for these requirements. —SS]

Personalization and Internationalization Covers UHR-PER-01 to UHR-PER-04, UHR-INT-01.
[Add the test cases for these requirements. —SS]

Learning Covers UHR-LRN-01 to UHR-LRN-04.
[Add the test cases for these requirements. —SS]

Understandability and Politeness Covers UHR-UND-01 to UHR-UND-03, UHR-POL-01 to UHR-POL-05.
[Add the test cases for these requirements. —SS]

Accessibility Covers UHR-ACC-01 to UHR-ACC-06.
[Add the test cases for these requirements. —SS]

4.2.3 Performance

These tests cover the Performance Requirements section of the SRS. Timing tests run on the reference phone, which is the Android phone the team uses for every timing measurement. Its model and Android version are recorded in the VnV Report so the results can be repeated. Names in sans serif are symbolic constants defined in the SRS, and tests use their values from there.

Speed and Latency Covers PR-SL-01 to PR-SL-04.

test-PR-SL-01	**App launch time**
Type	Dynamic, Manual
Initial State	The release build is installed on the reference phone, the user is signed in, and the app is fully closed (removed from recent apps).
Input/Condition	The tester opens the app from the home screen icon 20 times, closing it fully between each launch.
Output/Result	The home screen appears within 3 seconds in at least 19 of the 20 launches.
How test will be performed	The tester records each launch as a screen recording and measures the time from the icon tap to the first frame where the home screen is fully drawn. All 20 times are recorded in the VnV Report.

test-PR-SL-02	**Response to input**
Type	Dynamic, Automatic
Initial State	The release build is installed on the reference phone with a signed-in test account that has activity, goals, and a portfolio.
Input/Condition	A Detox script visits every main screen and presses the main buttons on each one, for at least 100 measured actions in total.
Output/Result	At least 95% of the measured transitions and button responses finish within 1 second.
How test will be performed	The script logs the time between each action and the moment the expected element on the next screen becomes visible. A summary script reports the share of actions under 1 second.

test-PR-SL-03	Trade confirmation time
Type	Dynamic, Automatic
Initial State	A signed-in test account has enough currency to buy, and some holdings to sell, in several assets.
Input/Condition	A Detox script places 50 buy orders and 50 sell orders of different sizes and confirms each one.
Output/Result	In at least 95% of the trades, the updated holdings and currency balance appear within 2 seconds of confirmation.
How test will be performed	The script records the time from the confirm tap until the new balance and holding quantity are shown, and checks that the shown values match the expected values after the trade.

test-PR-SL-04	Activity sync time
Type	Dynamic, Manual
Initial State	A test account is signed in on the reference phone. Health Connect holds new step and workout records that the app has not synced yet. The app is closed.
Input/Condition	The tester opens the app.
Output/Result	Within 10 seconds of opening, the currency balance includes the currency for all of the new Health Connect activity.
How test will be performed	The tester calculates the expected balance from the conversion rules beforehand, opens the app with a timer running, and records when the balance reaches the expected value. This is repeated 10 times with different amounts of new activity.

Safety-Critical Covers PR-SC-01 to PR-SC-08.

test-PR-SC-01	Daily earning limits
Type	Dynamic, Automatic
Initial State	A test account with no activity recorded for the test day.
Input/Condition	Activity records for one day at 0.5, 1, 1.5, 2, and 5 times HealthyDailyActivity, each sent to a fresh test account.
Output/Result	No account earns more than DailyEarnCap for the day. The currency earned per unit of activity above HealthyDailyActivity is lower than the rate below it.
How test will be performed	A JUnit test sends each set of records through the reward service and compares the earned amount against the cap and against the earned amount for the lower levels.

test-PR-SC-02	No penalties for missed days
Type	Dynamic, Automatic and Static
Initial State	A test account with a currency balance, holdings, and an active streak.
Input/Condition	(1) The account records no activity for several days. (2) The user freezes their streak before a missed day.
Output/Result	(1) The balance and holdings are unchanged after the inactive days. (2) The frozen streak is not reset by the missed day.
How test will be performed	A JUnit test advances the server clock over the inactive days and checks the balance, holdings, and streak. A code review of every place that writes to balances and holdings confirms that none of them lowers a value because of inactivity.

test-PR-SC-03	Simulated market notice
Type	Static, Manual inspection
Initial State	The release build with a test account that holds assets.
Input/Condition	The tester visits every screen that shows prices, holdings, or returns.
Output/Result	Each of these screens shows the notice that the market is simulated and that results do not predict real returns, and links to the explanation of what the simulation does not model.
How test will be performed	Two team members go through a checklist of all market and portfolio screens taken from the app's navigation. For each screen they record whether the notice is visible and whether the link opens the explanation.

test-PR-SC-04	Not financial advice statement
Type	Dynamic, Automatic and Static
Initial State	A fresh install of the app with no account.
Input/Condition	The tester tries to finish sign-up without acknowledging the statement, then acknowledges it and finishes sign-up, then opens settings.
Output/Result	Sign-up cannot continue until the statement is acknowledged, and the statement can be opened from settings.
How test will be performed	A Detox script checks the sign-up and settings behaviour. Separately, a team member reviews all lessons, tips, and portfolio insights against the list in the SRS and records any text that recommends a specific real-world investment. The test passes when none is found.

test-PR-SC-05	Exercise injury warning
Type	Dynamic, Automatic
Initial State	A fresh install of the app with no account.
Input/Condition	The tester tries to finish sign-up without acknowledging the injury warning and the terms of use, then acknowledges both, finishes sign-up, and opens settings.
Output/Result	Sign-up cannot continue until both are acknowledged, and both can be opened from settings.
How test will be performed	A Detox script runs the sign-up flow with each acknowledgement left out in turn, then checks that settings links to both.

test-PR-SC-06	Safe goal progression
Type	Dynamic, Automatic
Initial State	A test account with an automatically raised goal plan.
Input/Condition	(1) The goal service generates eight weeks of goals after the user meets every goal. (2) The user misses MissedGoalLimit goals in a row. (3) The user lowers a goal from the goal screen.
Output/Result	(1) No weekly goal is more than MaxGoalIncrease above the week before, and every week has at least one rest day. (2) The app suggests a lower goal. (3) The lower goal is saved and used.
How test will be performed	JUnit tests check the generated plan and the suggestion. A Detox script lowers a goal from the goal screen and checks the saved value.

test-PR-SC-07	Notification limits
Type	Dynamic, Automatic
Initial State	A test account with notifications on and quiet hours set.
Input/Condition	Events that would send goal-related notifications are created throughout a simulated day, including during quiet hours.
Output/Result	No notifications are sent during quiet hours, and no more than MaxDailyNotifications goal-related notifications are sent in the day. After the user raises the limit, the new limit is used.
How test will be performed	A JUnit test runs the notification scheduler with a fixed clock and counts the notifications it sends and the times it sends them.

test-PR-SC-08	Safe leaderboard rules
Type	Dynamic, Automatic
Initial State	A leaderboard group with several test users who have different goals and different amounts of activity.
Input/Condition	The leaderboard is computed, one user leaves the group, and the leaderboard is computed again. A new account is also created.
Output/Result	Rankings are ordered by each user's percentage of their goal, not by raw activity. The user who left no longer appears. No user's balance changes because of their ranking. The new account is not on any leaderboard until the user opts in.
How test will be performed	A JUnit test sets up users where ranking by raw activity and ranking by goal percentage give different orders, then checks the order, the membership, and every balance before and after.

Precision or Accuracy Covers PR-PA-01 to PR-PA-06.

test-PR-PA-01	Exact money values
Type	Dynamic, Automatic
Initial State	A test account with a known starting balance.
Input/Condition	A scripted sequence of 1,000 buy and sell trades at prices that do not divide evenly, such as 33.33 and 0.07.
Output/Result	The stored balance after the sequence matches the expected balance exactly, and every amount shown in the app has two decimal places.
How test will be performed	A JUnit test runs the trades through the trading service and compares the final balance with a value computed separately using exact decimal arithmetic. A Detox check reads the displayed balance and trade amounts and checks their format.

test-PR-PA-02	Consistent currency awards
Type	Dynamic, Automatic
Initial State	An empty test account for each activity record set.
Input/Condition	A fixed set of test activity records covering steps, workouts of each type, and edge values such as zero activity. Each set is processed twice.
Output/Result	The awarded currency matches the hand-calculated amount from the published conversion rules, and both runs award the same amount.
How test will be performed	The team calculates the expected amounts by hand and stores them as test fixtures. A JUnit test compares the reward service's output against them.

test-PR-PA-03	No double counting
Type	Dynamic, Automatic
Initial State	A test account with two connected sources.
Input/Condition	The same walk is reported by both sources with overlapping times. A second case has two separate walks, one from each source, that do not overlap.
Output/Result	The overlapping walk earns the amount for one recording. The two separate walks each earn currency.
How test will be performed	A JUnit test sends both cases through the import and reward services and compares the awards with the expected amounts.

test-PR-PA-04	Portfolio value
Type	Dynamic, Automatic
Initial State	Several test portfolios with different holdings, fixed simulated prices, and some uninvested currency.
Input/Condition	The portfolio screen is opened for each test portfolio.
Output/Result	The displayed value equals the sum of quantity times price for each holding, plus uninvested currency, to the cent.
How test will be performed	The expected values are calculated by hand. A JUnit test checks the value returned by the API, and a Detox script checks the value shown on screen.

test-PR-PA-05	Workout dates and late syncs
Type	Dynamic, Automatic
Initial State	A test account with an active streak.
Input/Condition	(1) A workout done at 11:30 pm is synced after midnight. (2) A workout recorded in a different time zone is synced. (3) A workout from two days ago arrives after the streak was already marked as broken.
Output/Result	Each workout is recorded on the day it happened on the source device. In case (3) the streak and the rewards for that day are recalculated to include the late workout.
How test will be performed	A JUnit test sends records with set source timestamps and time zones, using a fixed server clock, then checks the stored dates, the streak, and the rewards.

test-PR-PA-06	Realistic market simulation
Type	Dynamic, Automatic
Initial State	The market simulation with its calibrated settings and a fixed random seed.
Input/Condition	The simulation is run over at least 30 simulated years for a broad market asset, with several different seeds.
Output/Result	The average annual return and the annual volatility are consistent with historical data for a broad market index, and each run includes downturns.
How test will be performed	A script computes the average annual return, annual volatility, and largest drop for each run and compares them with the same measures from the historical data used for calibration. The acceptable ranges are agreed with the supervisor before the test and recorded in the VnV Report.

Robustness or Fault-Tolerance Covers PR-RFT-01 to PR-RFT-04.

test-PR-RFT-01	Offline activity
Type	Dynamic, Manual
Initial State	A test account signed in on the reference phone, synced, then put in airplane mode with the app closed.
Input/Condition	Activity is recorded in Health Connect over a gap of OfflineSyncWindow (set by moving the activity timestamps back, or by waiting out the period). The phone is then reconnected and the app is opened.
Output/Result	All of the activity from the gap is credited at the next sync.
How test will be performed	The tester compares the credited currency with the amount calculated by hand for the recorded activity.

test-PR-RFT-02	Atomic writes
Type	Dynamic, Automatic
Initial State	A test account with a known balance and holdings.
Input/Condition	(1) A failure is injected at each step of every multi-step write: trades, reward credits, and progress updates. (2) The connection or server is cut in the middle of a test trade. (3) 100 trade requests are sent at the same time against one account.
Output/Result	(1) Each failed write leaves the data as it was. (2) After the cut, the balance and holdings match either the state before the trade or the state after it, and a completed trade used the confirmed price. (3) Every request is either fully completed or fully rolled back, and the balance is never negative.
How test will be performed	JUnit tests use fault injection in the service layer to fail each step in turn. The concurrent test sends the 100 requests from parallel threads, then checks that the final balance and holdings equal the starting state plus the completed trades.

test-PR-RFT-03	Failing data sources
Type	Dynamic, Automatic
Initial State	A test account with two connected sources.
Input/Condition	One source is made unavailable. In a second run, it returns malformed data.
Output/Result	The app stays usable with the other source, retries the failed sync, then shows a message naming the failing source and the cause. No currency is awarded for the malformed data.
How test will be performed	The failing source is replaced with a mock that times out or returns malformed records. Backend tests check the retries and the awards, and a Detox script checks the app and the message.

test-PR-RFT-04	Backend unavailable
Type	Dynamic, Manual
Initial State	A test account that has synced at least once on the reference phone.
Input/Condition	The backend is stopped, then the app is opened.
Output/Result	The app opens without crashing, shows the last known balance and portfolio, and shows a notice that the data may be out of date.
How test will be performed	The tester stops the backend, opens the app, visits each main screen, and records whether any screen crashes or is empty.

Capacity Covers PR-C-01, PR-C-02.

test-PR-C-01	Concurrent users
Type	Dynamic, Automatic
Initial State	The backend is deployed to the test server with 500 registered test accounts.
Input/Condition	A load test runs 100 simulated users at the same time. Each one browses screens and places trades.
Output/Result	Response times during the load test meet PR-SL-02 and PR-SL-03.
How test will be performed	The load testing tool sends the API calls that the app makes for each action and records the response times. While the load runs, test-PR-SL-02 and test-PR-SL-03 are also run on the reference phone.

test-PR-C-02	Two years of history
Type	Dynamic, Automatic
Initial State	A test account loaded with two years of daily activity and weekly trades.
Input/Condition	The user opens the activity history, trade history, and progress screens and scrolls to the oldest records.
Output/Result	The full two years of history is shown, and the screens meet PR-SL-02.
How test will be performed	A seeding script creates the test data. A Detox script opens each history screen, checks that the oldest records are shown, and times each transition.

Scalability or Extensibility Covers PR-SE-01 to PR-SE-03.

test-PR-SE-01	Adding a fitness source
Type	Static, Code inspection
Initial State	The code with the fitness sources available at release.
Input/Condition	The team adds a mock fitness source.
Output/Result	All of the changes needed are inside the integration module, and the currency conversion code is unchanged.
How test will be performed	A team member adds the mock source on a branch. The reviewers check the diff to confirm that every changed file is inside the integration module.

test-PR-SE-02	Adding an asset type
Type	Static, Code inspection
Initial State	The code with stocks, bonds, and ETFs.
Input/Condition	The team adds a new test asset type.
Output/Result	The change only adds asset definitions and market engine configuration. The buy and sell code is unchanged, and the existing trading tests pass with the new asset type.
How test will be performed	A team member adds the asset type on a branch. The reviewers check the diff, and the trading tests are run with the new asset type.

test-PR-SE-03	Adding server capacity
Type	Dynamic, Automatic
Initial State	The backend is deployed with extra server capacity, and the app version is the same one used in test-PR-C-01.
Input/Condition	The load test from test-PR-C-01 is run with 200 simulated users.
Output/Result	Response times meet PR-SL-02 and PR-SL-03, with no change to the mobile app.
How test will be performed	The team adds server capacity, reruns the load test at double the load, and confirms that the app's version number is the same as in test-PR-C-01.

Longevity Covers PR-L-01, PR-L-02.

test-PR-L-01	Supported Android versions
Type	Static and Dynamic, Automatic
Initial State	The release build.
Input/Condition	(1) The target Android version in the build settings is compared with Google Play's requirement for new apps at release time. (2) The test suite is run on an emulator with the newest Android version available in April 2027.
Output/Result	The build meets Google Play's target version requirement, and the test suite passes on the newest Android version.
How test will be performed	A team member checks the build settings against Google Play's published requirement before release. The Detox suite is run on the newest Android emulator image before the final demo.

test-PR-L-02	Keeping user history
Type	Static, Code inspection
Initial State	The backend code, database schema, and server configuration.
Input/Condition	The team searches for every scheduled job, database event, and retention setting that deletes data.
Output/Result	None of them deletes activity or portfolio history for an active account. History is deleted only when the user deletes it or deletes their account.
How test will be performed	A team member lists every delete path in the code and database configuration. A second team member reviews the list and confirms that each one is started by the user.

4.2.4 Operational and Environmental

These tests cover the Operational and Environmental Requirements section of the SRS.

Productization Covers PROD-01, PROD-02.
[Add the test cases for these requirements. —SS]

4.2.5 Maintainability and Support

These tests cover the Maintainability and Support Requirements section of the SRS.

Maintenance Covers MAS-MNT-01 to MAS-MNT-05.
[Add the test cases for these requirements. —SS]

Supportability Covers MAS-SUP-01.
[Add the test cases for these requirements. —SS]

Adaptability Covers MAS-ADPT-01 to MAS-ADPT-03.
[Add the test cases for these requirements. —SS]

4.2.6 Security

These tests cover the Security Requirements section of the SRS (**?**), spanning access control, data and logic integrity, privacy, economic and security audit logging, and immunity against attacks.

Access Covers SEC-ACC-01 to SEC-ACC-05.

1. **T-SEC-ACC-01: Authenticated Access Token Validation**

   Type: Dynamic, Automated

   Initial State: Backend server running in test profile with test database; user account registered; valid access token issued with a 60-minute expiry window.

   Input/Condition: Send requests to a protected system resource (such as fitness activities or portfolio summaries) under four conditions: (1) missing authentication credentials, (2) tampered token signature, (3) expired token (timestamp beyond the 60-minute lifetime), and (4) authentic, unexpired token.

   Output/Result: Requests lacking credentials, with a tampered token, or with an expired token are rejected with HTTP 401 Unauthorized. The request with a valid token returns HTTP 200 OK with the requested user data.

   How test will be performed: Automated integration tests dispatching requests across token states against protected system services.

2. **T-SEC-ACC-02: Role-Based Authorization and Cross-User Isolation**

   Type: Dynamic, Automated

   Initial State: Two standard test users (User A and User B) with distinct portfolios and data records, and one administrator user.

   Input/Condition: (1) User A attempts to read User B's private portfolio. (2) User A attempts to execute a trade or modify records on

User B's behalf. (3) User A attempts to access system-wide administrative audit logs. (4) Administrator requests access to system-wide administrative audit logs.

Output/Result: Cross-user reads, cross-user modifications, and standard-user administrative requests are rejected with HTTP 403 Forbidden with zero data modifications. The administrative request is accepted (HTTP 200 OK) returning the requested audit records.

How test will be performed: Automated security integration tests verifying role-based access control rules and tenant isolation across system boundaries.

3. **T-SEC-ACC-03: Secure Credential Storage and Secret Exclusion**

   Type: Static and Dynamic, Automated

   Initial State: Repository source commit under evaluation; mobile test environment.

   Input/Condition: (1) Execute secret scanning on the complete version control repository history for API keys, private keys, and client secrets. (2) Store third-party fitness integration credentials via the client application and inspect local persistent device storage.

   Output/Result: (1) Repository secret scan detects zero exposed credentials or keys. (2) Integration credentials are encrypted at rest using platform-managed secure key storage (e.g., AES-256 GCM); zero cleartext tokens exist on local disk.

   How test will be performed: Automated repository secret scanning in CI and mobile client instrumentation tests inspecting device storage.

4. **T-SEC-ACC-04: Password Strength, Lockout, and Multi-Factor Authentication**

   Type: Dynamic, Automated

   Initial State: User authentication service active with test database; common password dictionary loaded in memory.

Input/Condition: (1) Submit account registration requests using short passwords (< 8 characters), common dictionary passwords, and a strong password (> 8 characters). (2) Submit 5 consecutive incorrect password attempts for a user account, followed immediately by a 6th attempt with the correct password within 15 minutes. (3) Advance system time past 15 minutes (or submit a password reset) and retry with valid credentials. (4) For an account with multi-factor authentication (MFA) enabled, submit valid credentials, followed by an invalid one-time password (OTP), and then a valid OTP.

Output/Result: Short and dictionary passwords return HTTP 400 Bad Request; strong passwords return HTTP 201 Created. The 6th consecutive failed attempt triggers account lockout, rejecting subsequent login attempts with HTTP 423 Locked (or HTTP 403 Forbidden). Login succeeds once the lockout duration elapses. MFA challenge rejects invalid OTPs (HTTP 401 Unauthorized) and grants session access only upon valid OTP submission.

How test will be performed: Automated integration tests verifying authentication policies, lockout timers, and MFA verification flows.

5. **T-SEC-ACC-05: Session Termination and Device Unlinking**

Type: Dynamic, Automated

Initial State: Active user session with valid access and refresh tokens; client device linked to user account.

Input/Condition: (1) Client issues a sign-out request with its registered device identifier. (2) Attempt subsequent resource requests using the revoked access token. (3) Attempt to submit activity data from the unlinked device. (4) Present a session token that has been inactive for greater than 30 days.

Output/Result: Sign-out completes successfully (HTTP 200 OK) and unlinks the device. Requests presenting revoked tokens, data submissions from unlinked devices, and expired inactive sessions are rejected (HTTP 401 Unauthorized or HTTP 403 Forbidden).

How test will be performed: Automated integration tests verifying session invalidation, token revocation repositories, and device link states.

Integrity Covers SEC-INT-01 to SEC-INT-03.

1. **T-SEC-INT-01: Activity Telemetry Plausibility and Deduplication**

   Type: Dynamic, Automated

   Initial State: Authenticated user account with 0 virtual currency.

   Input/Condition: Ingest four workout telemetry payloads: (1) sustained cadence of 280 steps/min (exceeding plausibility threshold of 250 steps/min), (2) overlapping workout intervals from two integrations, (3) duplicate submission of an identical workout external sync ID, and (4) normal plausible workout (120 steps/min, no overlap).

   Output/Result: (1) Implausible cadence payload marked `UNVERIFIED` with zero virtual currency awarded. (2) Overlapping interval flagged `UNVERIFIED`; only unique segments credited. (3) Duplicate payload rejected by idempotency key with zero duplicate currency minted. (4) Plausible workout verified and virtual currency awarded.

   How test will be performed: Automated unit and integration tests executing domain validation rules on activity telemetry payloads.

2. **T-SEC-INT-02: Server-Side Authority on Valuations and Trades**

   Type: Dynamic, Automated

   Initial State: Authenticated user with virtual currency balance; server-authoritative asset price set (e.g., 50.00 FLX for `FLEX_CORP`).

   Input/Condition: (1) Client submits a trade request specifying a client-dictated execution price (e.g., 0.01 FLX) and total cost. (2) Client submits a currency reward claim specifying an arbitrary client-dictated reward amount.

   Output/Result: (1) The system rejects the trade request with HTTP 400 Bad Request, or ignores the client-supplied price and executes strictly at the server-authoritative valuation (50.00 FLX). (2) The system rejects client-dictated reward amounts and calculates currency rewards strictly from verified workout metrics.

   How test will be performed: Automated integration tests verifying server-side pricing enforcement and reward calculation authority.

3. **T-SEC-INT-03: Database Backup Integrity and Restoration Drill**

   Type: Dynamic Operational Verification and Static Configuration Review

   Initial State: Staging database populated with seed dataset of users, portfolios, and audit records; automated backup cron job configured.

   Input/Condition: (1) Inspect cloud backup policy scripts for 24-hour backup interval and 30-day retention lifecycle. (2) Trigger an automated backup snapshot, record SHA-256 data checksums, restore onto a clean containerized database instance, and measure restoration time.

   Output/Result: Configuration confirms daily backups and 30-day automated retention. Database restores completely within target limit (< 15 minutes in test environment), with record counts and checksums matching 100%.

   How test will be performed: Automated staging script executing backup creation, containerized restoration, and data integrity verification.

Privacy Covers SEC-PRV-01 to SEC-PRV-03.

1. **T-SEC-PRV-01: Encryption in Transit and at Rest**

   Type: Dynamic Security Verification and Database Inspection

   Initial State: Staging backend deployed behind reverse proxy; database running with encrypted tablespace.

   Input/Condition: (1) Execute network handshake scan against API endpoints testing plain HTTP and TLS protocols 1.0, 1.1, 1.2, and 1.3. (2) Insert sensitive biometric fields (heart rate, body weight) into database and inspect raw storage files on disk.

   Output/Result: (1) Plain HTTP requests and TLS connections below version 1.3 are rejected; only TLS 1.3 handshakes succeed. (2) Direct inspection of database disk blocks reveals AES-256 ciphertext; sensitive biometric data is unreadable in raw form.

   How test will be performed: Automated script combining TLS protocol scanning and raw disk inspection on test database volumes.

2. **T-SEC-PRV-02: Private by Default and Leaderboard Opt-In Isolation**

   Type: Dynamic, Automated

   Initial State: Newly registered user with logged workouts and simulated portfolio; default settings unchanged.

   Input/Condition: (1) Query public user profile and public leaderboard for the new user. (2) User explicitly opts in to leaderboard sharing with a public alias. (3) User toggles leaderboard sharing off.

   Output/Result: (1) By default, public profile returns 404 or minimal stub; user is omitted from public leaderboards. (2) Opted-in leaderboard displays only chosen alias and aggregate score; real name, email, and biometric metrics remain private. (3) Opting out removes the user from public leaderboards immediately.

   How test will be performed: Automated API integration tests verifying endpoint responses across default, opt-in, and opt-out states.

3. **T-SEC-PRV-03: User Account Deletion and PII Eradication**

   Type: Dynamic, Automated

   Initial State: User account populated with profile, health logs, active session, and transactions in the financial audit ledger.

   Input/Condition: User submits an account deletion request with credential confirmation. Inspect active application databases, attempt login with deleted credentials, and inspect the financial audit ledger.

   Output/Result: All personal identifying information (name, email, device identifiers) and health logs are purged from the active system within 24 hours. Subsequent login attempts are rejected (HTTP 401 Unauthorized). The financial audit ledger preserves transactional continuity but replaces user identifiers with an irreversible pseudonym.

   How test will be performed: Automated end-to-end integration test asserting database purge, login denial, and ledger anonymization.

Audit Covers SEC-AUD-01, SEC-AUD-02.

1. **T-SEC-AUD-01: Economic Event Ledger Immutability**

   Type: Dynamic Database Privilege Verification

   Initial State: Backend connected to database using standard application credentials; sample rows present in `economic_audit_ledger`.

   Input/Condition: (1) Record simulated trade or reward via standard service API. (2) Attempt direct SQL `UPDATE` on an existing ledger record. (3) Attempt direct SQL `DELETE` on an existing ledger record.

   Output/Result: Service API inserts new audit record with immutable timestamp. Direct SQL `UPDATE` and `DELETE` commands are denied by database role permissions with error 1142 (command denied); ledger records remain unmodified.

   How test will be performed: Automated database integration test executing restricted SQL commands using application service credentials.

2. **T-SEC-AUD-02: Structured Security Event Logging**

   Type: Dynamic, Automated

   Initial State: Log appender active and configured for structured JSON output.

   Input/Condition: Trigger security events: 3 failed login attempts, activity payload with tampered signature, rate limit exceeded ($>$ 60 req/min), and unhandled server exception.

   Output/Result: Logs capture structured JSON entries containing severity level (WARN/ERROR), ISO 8601 UTC timestamp, client IP, event type, and user ID (or anonymous). Zero sensitive credentials or plaintext passwords appear in log records.

   How test will be performed: Automated integration test with in-memory log appender asserting on serialized JSON log structure and contents.

Immunity Covers SEC-IMM-01 to SEC-IMM-03.

1. **T-SEC-IMM-01: Injection Defense (SQLi, XSS, Command Injection)**

Type: Static SAST and Dynamic DAST Penetration Testing

Initial State: Backend service deployed in containerized test environment with database connectivity.

Input/Condition: (1) Run static analysis on system source code for unparameterized queries, unescaped reflection, and command execution. (2) Execute automated penetration scanning dispatching SQL injection, cross-site scripting (XSS), and command injection attack vectors across all external system interfaces.

Output/Result: Static analysis reports zero high-severity injection vulnerabilities. Penetration scanning reports zero exploitable injection vulnerabilities; malicious payloads are sanitized or rejected (HTTP 400/422).

How test will be performed: Automated security test pipeline combining static code analysis (SAST) and dynamic penetration scanning (DAST).

2. **T-SEC-IMM-02: API Rate Limiting Enforcement**

   Type: Dynamic, Automated

   Initial State: Rate limiting filter configured with threshold of 60 requests/minute per client IP.

   Input/Condition: Client sends 60 consecutive requests within 30 seconds, sends 10 additional requests within the remainder of the 60-second window, waits 65 seconds, and sends a subsequent request.

   Output/Result: Requests 1 to 60 process normally. Requests 61 to 70 return HTTP 429 Too Many Requests with a `Retry-After` header. The request following the cooldown period processes normally.

   How test will be performed: Automated integration test using a multi-threaded HTTP client dispatching timed request bursts.

3. **T-SEC-IMM-03: Client Environment Tampering and Root Detection**

   Type: Dynamic Verification

Initial State: Android test device or emulator with root privileges enabled (`su` binary present, test keys installed).

Input/Condition: (1) Deploy production release build onto rooted device and attempt app launch, fitness sync, and leaderboard access. (2) Deploy debug build onto rooted device and attempt same actions.

Output/Result: Production build detects untrusted environment, displays warning modal, and disables virtual currency minting and leaderboard features. Debug build runs without restrictions for testing.

How test will be performed: Automated Android UI test and manual release checklist verifying root detection library signals.

Security Review Walkthrough Static reviews are conducted once per milestone release cycle (Rev 0, Final Demo) by the Backend Lead (Daniel Franze-Da Silva), Security and Documentation Lead (Arjun Karthik), and an independent peer reviewer. The team reviews: (1) ISO/IEC 27001 Annex A mapping against system controls (COMP-STD-07), (2) least privilege scopes for third-party fitness APIs (SEC-ACC-03), (3) repository secret scan results, and (4) dependency vulnerability reports ensuring zero unresolved High/Critical CVEs.

4.2.7 Cultural

These tests cover the Cultural Requirements section of the SRS.

Cultural Covers CUL-01, CUL-02.
[Add the test cases for these requirements. —SS]

4.2.8 Compliance

These tests cover the Compliance Requirements section of the SRS.

Legal Covers COMP-LEG-01 to COMP-LEG-07.
[Add the test cases for these requirements. —SS]

Standards Compliance Covers COMP-STD-01 to COMP-STD-07.
[Add the test cases for these requirements. —SS]

4.2.9 User Documentation and Training

These tests cover the User Documentation and Training Requirements section of the SRS (**?**), spanning in-app educational documentation, setup guides, help systems, and user training onboarding mechanisms.

User Documentation Covers DOC-01 to DOC-04.

1. **T-DOC-01: Financial and Fitness Glossary Presence**

   Type: Static Inspection

   Initial State: Application build running on a test device; glossary view accessible in-app.

   Input/Condition: Open the in-app glossary and search for required educational terms (compound interest, inflation, ETFs, bond yields, and portfolio diversification).

   Output/Result: Binary: Pass if the searchable glossary is present and contains entries for all required terms; Fail otherwise.

   How test will be performed: Manual walkthrough inspection of the glossary view verifying term presence and search functionality.

2. **T-DOC-02: Fitness Integration Guide Presence**

   Type: Static Inspection

   Initial State: Application build running on a test device; integration settings or help section open.

   Input/Condition: Navigate to the fitness integration configuration section and open the setup guide.

   Output/Result: Binary: Pass if illustrated setup instructions with screenshots are present for all supported fitness providers (e.g., Garmin, Android Health Connect) and include troubleshooting guidance; Fail otherwise.

   How test will be performed: Visual inspection confirming presence and completeness of illustrated setup and troubleshooting instructions.

3. **T-DOC-03: Searchable Help Center Presence**

   Type: Dynamic, Semi-Automated

   Initial State: Application open to the settings menu.

   Input/Condition: Open the in-app help center from settings and enter search queries covering core topics (account management, activity sync, rewards, portfolio, simulated market, virtual currency, privacy).

   Output/Result: Binary: Pass if the in-app help center opens without an external browser, contains articles covering all required topics, and returns keyword search results in under 200 ms; Fail otherwise.

   How test will be performed: Manual walkthrough and automated UI test timing keyword search queries and asserting topic article presence.

4. **T-DOC-04: Digital Manual and Release Notes Presence**

   Type: Static Inspection

   Initial State: Project repository on GitHub checked out at latest milestone commit.

   Input/Condition: Inspect the documentation repository directory for the digital user manual and milestone release notes.

   Output/Result: Binary: Pass if a self-contained user manual (PDF and Markdown formats) documenting architecture, navigation, data flows, and troubleshooting, along with milestone release notes, is present in the repository; Fail otherwise.

   How test will be performed: Peer-review file audit by team members verifying presence and completeness of documentation artifacts in the repository.

Training Covers TRN-01 to TRN-03.

1. **T-TRN-01: Interactive Onboarding Walkthrough Presence**

   Type: Dynamic, Manual

Initial State: Clean application state immediately after creating a new user account.

Input/Condition: Complete new account registration to trigger onboarding; inspect the four-step walkthrough (goals, rewards, simulated investing, compounding chart), test the "Skip Tour" option, and relaunch the walkthrough from settings.

Output/Result: Binary: Pass if the four-step walkthrough is present, takes under 90 seconds to complete, provides a visible "Skip Tour" button on every step, and is re-accessible from settings; Fail otherwise.

How test will be performed: Manual timed walkthrough verifying the presence of all four tour steps, skip buttons, and settings re-entry.

2. **T-TRN-02: Progressive Feature Discovery Presence**

   Type: Dynamic, Automated

   Initial State: Newly registered user account with zero trades and zero logged physical activity days.

   Input/Condition: Navigate to trading and portfolio analytics screens as a novice user; then simulate logging three days of physical activity or executing an initial trade, and re-examine the screens.

   Output/Result: Binary: Pass if advanced trading indicators and portfolio analytics remain hidden or collapsed initially and are revealed once the activity or trade threshold is met; Fail otherwise.

   How test will be performed: Automated Compose UI test asserting view hierarchy presence and visibility states of advanced features before and after threshold completion.

3. **T-TRN-03: Demonstrator Quick-Start Kit Presence**

   Type: Dynamic Operational Verification

   Initial State: Application development/staging build running; demo seeding script available.

   Input/Condition: Execute the automated demo seeding script or trigger the demonstrator debug toggle.

Output/Result: Binary: Pass if the demonstrator guide is present and the seeding script loads 30 days of simulated activity and portfolio history within 10 seconds; Fail otherwise.

How test will be performed: Execution of the demonstrator script or debug toggle timing dataset initialization and verifying data display on trading and compounding charts.

4.3 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

The table below maps each system test to the SRS requirement whose fit criterion it checks. The last column lists other requirements that the test also measures, such as load tests that rerun the speed checks. Rows for the remaining requirement sections are added as their test cases are written.

Test Case	**SRS ID**	**Requirement**	**Also Supports**
test-PR-SL-01	PR-SL-01	App launch time	—
test-PR-SL-02	PR-SL-02	Response to input	—
test-PR-SL-03	PR-SL-03	Trade confirmation time	—
test-PR-SL-04	PR-SL-04	Activity sync time	—
test-PR-SC-01	PR-SC-01	Daily earning limits	—
test-PR-SC-02	PR-SC-02	No penalties for missed days	—
test-PR-SC-03	PR-SC-03	Simulated market notice	—
test-PR-SC-04	PR-SC-04	Not financial advice statement	—
test-PR-SC-05	PR-SC-05	Exercise injury warning	—
test-PR-SC-06	PR-SC-06	Safe goal progression	—
test-PR-SC-07	PR-SC-07	Notification limits	—
test-PR-SC-08	PR-SC-08	Safe leaderboard rules	—

Test Case	SRS ID	Requirement	Also Supports
test-PR-PA-01	PR-PA-01	Exact money values	—
test-PR-PA-02	PR-PA-02	Consistent currency awards	—
test-PR-PA-03	PR-PA-03	No double counting	—
test-PR-PA-04	PR-PA-04	Portfolio value	—
test-PR-PA-05	PR-PA-05	Workout dates and late syncs	—
test-PR-PA-06	PR-PA-06	Realistic market simulation	—
test-PR-RFT-01	PR-RFT-01	Offline activity	—
test-PR-RFT-02	PR-RFT-02	Atomic writes	—
test-PR-RFT-03	PR-RFT-03	Failing data sources	—
test-PR-RFT-04	PR-RFT-04	Backend unavailable	—
test-PR-C-01	PR-C-01	Concurrent users	PR-SL-02, PR-SL-03
test-PR-C-02	PR-C-02	Two years of history	PR-SL-02
test-PR-SE-01	PR-SE-01	Adding a fitness source	—
test-PR-SE-02	PR-SE-02	Adding an asset type	—
test-PR-SE-03	PR-SE-03	Adding server capacity	PR-SL-02, PR-SL-03
test-PR-L-01	PR-L-01	Supported Android versions	—
test-PR-L-02	PR-L-02	Keeping user history	—

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

Initial State:

Input/Condition:

Output/Result:

How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?